

How Much Information Fits in a Vector?

Anonymous authors

Paper under double-blind review

Abstract

Recent work in neural network interpretability has suggested that hidden activations of some deep models can be viewed as linear projections of much higher-dimensional vectors of sparse latent “features.” In general, this kind of representation is known as a superposition code. This work presents an information-theoretic account of superposition codes in a setting applicable to interpretability. We show that, when the number k of active features is very small compared to the number N of total features, simple decoding methods currently used by sparse autoencoders are reliable if d is a constant factor greater than the Shannon limit. Specifically, when $\ln k / \ln N \leq \eta < 1$ and H is the entropy of the latent vector in bits, asymptotically it suffices that $d/H > C(\eta)$ for a certain increasing function $C(\eta)$. However, $C(\eta)$ varies significantly depending on the decoding method used. For example, when $\eta = 0.3$, we show that the popular method of top- k thresholding typically requires a factor of $C = 4$ dimensions per bit. On the other hand, we introduce a family of methods with comparable computational requirements (the “matching- k algorithm”) that only require $C < 2$ even as η approaches 0.4. Our results address a lack of practical references on superposition codes in a very sparse regime.

Introduction

If each neuron in given neural network coded for a “meaningful feature” of its input, we could hope to reverse-engineer the network’s behavior on a neuron-by-neuron basis. However, many large models have been found to learn neurons that correlate simultaneously with apparently unrelated features. This phenomenon is known as polysemanticity. (See for example Nguyen et al. (2016), Zhang & Wang (2023) and Olah et al. (2020).)

The appearance of so-called “polysemantic neurons” is not surprising from a connectionist viewpoint. Since at least the 1980s, proponents of artificial neural networks have argued that these systems may naturally use **distributed representations**—coding schemes where individual features are represented by patterns spread over many units of computation, and conversely where each units carries information on many features. (This term was apparently coined in Rumelhart et al. (1986), Chapter 3.) In contrast, a *local*

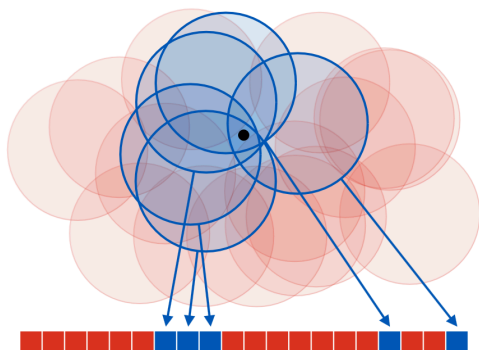


Figure 1: A coarse code representing a point on a plane. Each “neuron,” drawn as a red or blue square, encodes whether the point belongs to an associated “receptive field.” Although no neuron gives specific information on the position of the point, the overall code determines its position with reasonable accuracy.

representation would dedicate each unit to a single feature. See Thorpe (1989) for a general discussion of local and distributed codes. Figure 1 illustrates a classic example of a coarse code, one kind of distributed representation.

As of yet, relatively little is known about how large neural networks learn to represent information in their hidden layers or to what extent this information can be interpreted. However, should “interpretable features” exist, the connectivist viewpoint makes it natural that they would be stored with non-local codes. This is a common assumption in interpretability research today; for example, when Meng et al. (2022) intervened on an MLP layer of a language model to “edit” a factual association, both the “subject” and the “fact” were modeled as vectors of activations rather than individual channels.

To infer some kind of latent features from an activation vector y , one simple proposal is to model y as a linear projection $y = Fx$ of high-dimensional and *sparse* vector x . This idea is based on the remarkable fact that, for certain families of matrices F and for certain constraints on the sparsity of x , a d -dimensional linear image y codes uniquely for an N -dimensional sparse vector x when $N = \Omega(e^d)$.

We refer to the columns of F as codewords and the whole matrix F as a dictionary. Since y is a linear superposition of codewords, it is called a **superposition code** for x . The task of inferring x from a superposition code is known as sparse reconstruction, and the task of inferring the dictionary F from a distribution over the codes y is called dictionary learning. Both of these problems have been studied in the field of compressive sensing, although with different applications in mind. (See Elad (2010) for a review of classic work in the context of signal and image processing.)

Already in 2015, Faruqui et al. (2015) used a dictionary learning method to derive sparse representations for word embeddings and argued that these latents were more interpretable than the original embedding dimensions. More recently, a series of works beginning with Yun et al. (2021) have applied dictionary learning to the internal representations of transformer-based language models. Cunningham et al. (2023) suggested the use of **sparse autoencoders** (SAEs) and Templeton et al. (2024); Gao et al. (2024) scaled sparse autoencoders to production-size large language models.

Templeton et al. (2024) showed that latent features learned by SAEs are often highly interpretable, and that intervention on these features allows “steering” language models in predictable ways. However, as reported in Gao et al. (2024), even SAEs with extremely large numbers of latents suffer from an apparently irreducible reconstruction error. According to Sharkey et al. (2025), understanding the limitations of SAEs—and dictionary learning in general—is an important open question in the research program of mechanistic interpretability.

Contributions

To infer a latent representation $x \in \mathbb{R}^N$ from an activation vector $y \in \mathbb{R}^d$, sparse autoencoders use a simple estimate of the form $\hat{x}(y) = \sigma(Gy)$ where $G: \mathbb{R}^{d \times N}$ is a learnable matrix and σ is some kind of thresholding operation. Throughout this paper, we refer to any estimate of this form as a “one-step estimate” for x , since it requires only one step of computation. Gao et al. (2024), a work representative of research on large-scale sparse autoencoders, considered latent vectors with dimension N on the order of one million the number of non-zero coefficients k in each latent vector $\hat{x}$ was around one hundred.

Research on sparse autoencoders is mired by many kinds of scientific uncertainty. Of course, it is not known a priori how well activation vectors can be effectively modeled as sparse superposition codes. (It is not even necessarily clear what is meant by an “interpretable feature,” as per (cite something)). Furthermore, even if some activity of a model can be modeled as a superposition code, it’s not obvious whether the latent features being encoded can be recovered by a one-step estimate. Indeed, the compressive sensing literature provides many iterative methods for sparse recovery that require more computation but succeed more reliably. To what extent are the limitations of SAEs explained by the limitations of our sparse recovery methods?

One way to pose this question is to ask *how much information* we can expect a superposition code to hold. Classically, this kind of question is answered by the tools of information and coding theory. Given a “channel” with certain characteristics—for example, a band-limited telephone connection or a binary storage device

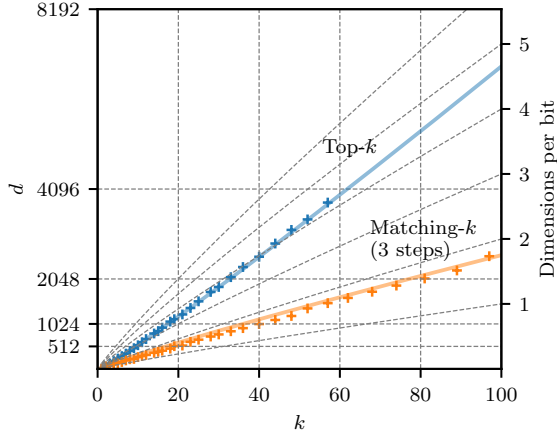


Figure 2: Crosses give embedding dimensions d required for two different methods to completely decode a k -sparse superposition code with empirical probability 0.95, and colored lines give rules of thumb derived from our theoretical discussion. The inverse “bitrate” $d/\tilde{H}_2$, where $\tilde{H}_2 = k \log_2(eN/k) \approx \log_2 \binom{N}{k}$, is indicated on the right axis. Top- k is a method currently used by sparse autoencoders, and three-step matching- k is a new method introduced in this work.

with a certain failure rate—the amount of information we can encode is asymptotically characterized by a channel “capacity” measured in bits per unit of channel usage.

The goal of this paper is to perform a similar analysis for sparse superposition codes in a setting that is meaningful to sparse autoencoders. Specifically, we consider Our analysis focuses on a specific toy example described in Section 1. Our main contributions are the following.

1. Asymptotically in a regime of sublinear sparsity, we prove that one-step estimates used by sparse autoencoders can reliably decode superposition codes $y \in \mathbb{R}^d$ so long as d is a constant factor greater than the Shannon limit (Proposition 4). One expression of our asymptotic result leads to an accurate rule of thumb for the empirical performance of the popular top- k method.
2. We introduce a simple extension to top- k which we call the **matching- k algorithm**. For the same value of (N, d) , matching- k only requires a small constant factor more computation to evaluate, but can decode superposition codes carrying more than two times as much information.

A concrete result of our findings is illustrated in Figure 2. For $k \in \{1, \dots, 100\}$ we show the number of dimensions d required for top- k and matching- k to accurately read a k -sparse subset of $\{1, \dots, 2^{20}\}$ that has been transmitted as a d -dimensional superposition. (The dictionary F is randomly generated; see Section 1 for more details.) As k increases, the ratio d/H of dimensions per bit of entropy required for top- k to perform reliably increases. For moderate values of k , top- k requires more than 4 dimensions per bit. In contrast, three-step matching- k requires only two dimensions per bit, while requiring only around 3 times more FLOPs for the same values of (N, d) .

Although our analysis is limited to a simple example, our results suggest that

Related Work

Here: some more on related work.

- Bajwa et al. (2010)
- Takeuchi (2024)
- Donoho et al. (2009)
- Donoho & Tanner (2009)

Structure of This Work

The body of this work is structured as follows.

- Section 1 introduces the toy superposition code to be studied and the decoding methods we will consider, including the matching- k algorithm.
- Section 2 briefly recalls the idea of a Shannon limit and explain two different heuristic predictions for the number of dimensions that may be required for a sparse superposition code to be decodable.
- Section 3 gives our main theoretical results on the performance of threshold decoding and our experimental comparison with matching- k .
- Finally, Section 4 considers the question of whether

1 Superposition Codes and Sparse Recovery

We begin by describing the toy scenario to be studied. Given a large number N , consider a map F that represents each subset $x \subseteq [N] = \{1, \dots, N\}$ by a linear combination

$$y(x) = \sum_{i \in x} f_i \in \mathbb{R}^d,$$

where the vectors $\{f_i \in \mathbb{R}^d : i \in [N]\}$ are chosen in advance and where the dimension d of the encoding is expected to be much smaller than N . The vectors f_i are called codewords for the elements of $[N]$, the collection $\{f_1, \dots, f_N\}$ is called a dictionary, and the image Fx is called a superposition code. It is useful to view x as a vector in $\{0, 1\}^N$ with coefficients

$$x_i = \begin{cases} 1 & : i \in x \\ 0 & : \text{otherwise} \end{cases}$$

and view F the matrix of column vectors $[f_1 \dots f_N]$, so that $y = Fx$ is a linear equation. In this work, we'll model our subset as a random variable X uniformly distributed over all subsets of some fixed size $k \ll N$. Keeping with the orders of magnitude discussed in Gao et al. (2024), we consider $N \approx 1,000,000$ and $k \approx 100$.

The problem of recovering a sparse signal x from a linear projection Fx is known as sparse recovery. This problem is considered in the subject of compressive sensing. In this work, we will use language from coding theory and think of x as some data to be encoded and the projection $y = Fx$ as a code. Thus, sparse recovery becomes the task of “decoding” x . Note that this is opposite to the convention used in sparse autoencoders, where the map estimating x from y is called the “encoder.” (In classical applications of autoencoders, the learned representation typically is understood as an efficient code for the data distribution, and typically has smaller dimension than the variable being modeled.)

Due to a connection with sparse linear regression, the subproblem of identifying the support of x from the projection Fx is often called model selection. When the coordinates of x take values in $\{0, 1\}$, sparse recovery reduces to a problem of model selection. In general, note that model selection presents the main difficulty of sparse recovery; once the support of x identified, its non-zero coordinates can be estimated easily e.g. by solving a least squares problem.

In practice, a dictionary F employed by a neural network will likely have special properties related to the nature of the data being encoded and the operations that need to be performed. However, for the purposes of encoding a uniformly random subset of $[N]$, one natural way to choose a dictionary is to make each codeword an independent random vector. The simplest motivation for this strategy is the fact that, so long as $d = \Omega(\ln N)$, we can expect a dictionary of N randomly chosen vectors to be a “nearly orthonormal” frame in the following sense.

Proposition 1 For $\mu > 0$ and $p > 0$, let $d > 2\mu^{-2}(2\ln N + \ln p^{-1})$, and let $\{F_1, \dots, F_N\}$ be independent draws from a uniform distribution over the unit sphere in $\mathbb{R}^N$. Then

$$\forall i \neq j, |\langle F_i, F_j \rangle| < \mu \quad (1)$$

with probability at least $(1 - p)$.

The infimum of all constants μ satisfying the condition (1) is often called the mutual coherence of a dictionary. Bounds on mutual coherence give a relatively simple way to guarantee the success of various sparse recovery algorithms. (For example, see Chapter 4 of Elad (2010).) In our scenario, we can say the following.

Remark 1 Let $x \subseteq [N]$ be a k -sparse subset. Given a dictionary F with mutual coherence smaller than $1/2k$, for any k -sparse vector $x \in \mathbb{R}^N$ and for all $i = 1, \dots, N$ we have

$$\forall i, x_i = \begin{cases} 1 : \langle f_i, Fx \rangle \geq 1/2 \\ 0 : \text{otherwise.} \end{cases}$$

Altogether we find that, for fixed k , it suffices to take $d = \Omega(\ln N)$ in order for each coefficient x_i to be retrievable in a very simple way. However, we will see that the minimum embedding dimension d that can

We informally refer to decoding methods that require only a single “step” of estimation and thresholding as **one-step decoders**. In contrast, the field of compressive sensing provides a number of more elaborate *iterative* methods for sparse recovery. However, many of these methods are not applicable to the very large problems faced by sparse autoencoders, where performing a training run using only a one-step method is already computationally demanding. In this work, we consider a family of relatively cheap iterative algorithms which we call **matching- k decoders**.

1.1 MAP Thresholding

One way to motivate the construction of one-step estimates is to think of the isolated subproblem of estimating a coefficient X_i from the model

$$Y = X_i f_i + \overbrace{\sum_{j \neq i} X_j f_j}^{Z_i}. \quad (2)$$

(Here we write X and Y with capital letters to emphasize they are random variables.) If we approximate the sum Z_i of other codewords as centered Gaussian noise with covariance Σ , the inference problem for X_i becomes tractable; specifically, if we define an inner product by $\langle v, w \rangle_\Sigma = \frac{1}{2} v^T \Sigma^{-1} w$, then the posterior on X_i can be expressed in terms of the linear function

$$\lambda_i(Y) = \frac{\langle f_i, Y \rangle_\Sigma}{\|f\|_\Sigma^2}.$$

In signal processing, this is called a matched filter for the codeword f_i . In terms of the matched filter,

$$\ln P(X_i = x | Y = y) = \frac{\rho_i}{2} (x - \lambda_i(y))^2 + \ln P(X_i = x) + C \quad (3)$$

where $\rho_i = (\text{Var } \lambda_i(Z_i))^{-1}$ is called the “signal-to-noise ratio.” (See Appendix B for a review.)

When f_i are independent draws from some isotropic distribution, the covariance Σ can be modeled as a multiple of the identity. In this case we have $\lambda_i(Y) = \langle f_i, Y \rangle / \|f_i\|$, and the maximum a posteriori (MAP) estimate is for X_i is simply

$$\hat{x}_i(y) = \begin{cases} 1 : \frac{\langle f_i, y \rangle}{\|f_i\|} \geq \tau \\ 0 : \text{otherwise} \end{cases} \quad (4)$$

where

$$\tau = \frac{1}{2} + \rho_i^{-1} \ln \frac{P(X_i = 1)}{P(X_i = 0)}.$$

In general, we refer to the estimate (4) as threshold decoding at level τ .

The quantities ρ_i be estimated by computing the variance under a random dictionary F ; since $\text{Var} \langle F_i, F_j \rangle = 1/d$ for a pair of independent uniform draws from the unit sphere in $\mathbb{R}^d$, we have

$$\begin{aligned} \text{Var}_{F,X} \langle F_i, Z_i \rangle &= P(X_i = 1) \text{Var}(\langle F_i, Z_i \rangle | X_i = 1) + P(X_i = 0) \text{Var}(\langle F_i, Z_i \rangle | X_i = 0) \\ &= \frac{k(k-1) + (N-k)k}{Nd} = \frac{k-k/N}{d} \approx \frac{k}{d} \end{aligned}$$

Putting $\rho_i^{-1} = d/k$ in (4) gives a concrete one-step estimate which we call MAP threshold decoding.

Now, if the number k of non-zero latents is known or can be guessed in advance, one simple improvement is to perform thresholding on the basis of this information—that is, to set $\hat{x}_i(y) = 1$ for the k indices i where the filters $\lambda_i(y)$ are maximized. We call this top- k decoding. Note that in some works (Elad (2010)) top- k is known as the thresholding algorithm.

1.2 Matching- k

While one-step methods are fast, iterative algorithms from compressive sensing often achieve better recovery. However, methods like orthogonal matching pursuit that select features one at a time are computationally prohibitive for sparse autoencoders. We consider a middle ground called matching- k that selects features in batches over a small number of steps.

The algorithm divides the task of selecting k features into T steps, choosing $k_1, k_2, \dots, k_T$ features in each step where $\sum_{t=1}^T k_t = k$. At each step, we compute scores based on the residual signal, select the top- k_t features, and subtract their contribution before proceeding.

Input: Signal $y \in \mathbb{R}^d$, dictionary $F \in \mathbb{R}^{d \times N}$, sparsity k , steps T

Output: Sparse vector $\hat{x} \in \{0, 1\}^N$

Choose step sizes $k_1, \dots, k_T$ with $\sum_t k_t = k$ and $\max_t k_t$ minimized ;

$\hat{x} \leftarrow \mathbf{0}$;

for $t = 1$ **to** T **do**

$r \leftarrow y - F\hat{x}$;

$S \leftarrow$ top- k_t largest coordinates of $F^T r$;

$\hat{x}_S \leftarrow 1$;

end

return $\hat{x}$

// Update residual

// Detect k_t new codewords

// Update reconstruction

Algorithm 1: Matching- k algorithm

When $T = 1$, matching- k reduces to top- k decoding. When $T = k$, selecting one feature per step, it becomes orthogonal matching pursuit. For intermediate values like $T = 2$ or $T = 3$, the algorithm provides a practical compromise between speed and accuracy.

2 Information Theory Bounds

Suppose that the latent variable Y to be encoded is a symbol drawn from an alphabet of size N . What is the minimum dimension d such that each value of Y can be represented by a vector $X \in \mathbb{R}^d$?

When the vector X can be stored exactly, the answer to our question simply depends on the number of values it can take. If the coefficients of X are 16 bit floating point numbers, then each coefficient can store (nearly) 2^{16} distinct numbers, and overall we need only about $d_{\min} \approx \frac{1}{16} \log_2 N$ dimensions.

However, we expect that activation vectors within real neural networks are subject to various kinds of interference. For example, an activation vector X may need to represent the symbol Y while also holding information on another latent quantity, or it may be subject to explicitly induced noise like dropout. Characterizing the “capacity” of a vector in the presence of interference is a basic problem in coding theory.

One common model is to consider that X is subject to additive white Gaussian noise. More specifically, we let $Z \in \mathbb{R}^d$ be a vector of independent Gaussians each of variance P and consider the problem of recovering Y from $X + Z$. The following is then a typical (but remarkable) result of information theory.

Proposition 2 (A Shannon limit) *Let the random variable $X \in [N]$ be uniformly distributed and let $Z \in \mathbb{R}^d$ be independent Gaussian noise of variance P . Suppose there exists a pair of maps $F: [N] \rightarrow \mathbb{R}^d$ and $G: \mathbb{R}^d \rightarrow [N]$ so that*

$$G(F(X) + Z) = X$$

with probability at least $(1 - p)$, and suppose the variance of each coordinate of $F(X)$ is bounded by 1. Then

$$d \geq 2 \frac{(1 - p) \ln N - \ln 2}{\ln(1 + P^{-1})}.$$

Asymptotically for large N , this means that

$$\frac{d}{H} \geq (1 + o(1)) \frac{2}{\ln(1 + P^{-1})}$$

if there is any pair (F, G) that transmits X with probability greater than 0, where $H = \ln N$ is the entropy of X . In other words, transmitting at least $2/\ln(1 + P^{-1})$ dimensions per nat.

Conversely, it can be shown that our bound on the ratio d/H is asymptotically tight. More specifically, for any $\epsilon > 0$ and for sufficiently large N , it is enough to take

$$\frac{d}{H} \geq (1 + \epsilon) \frac{2}{\ln(1 + P^{-1})}$$

for there to exist some pair (F, G) satisfying the conditions of Proposition 2 with p arbitrary close to 0. For a reference on this fact, see Chapter 9 of Cover & Thomas (2006).

In the remainder of this work, we are interested in applying a similar point of view to our toy superposition coding problem. Although we do not consider the influence of noise, the minimum dimension d required for a superposition to store a k -element subset of $[N]$

Let’s begin by considering a superposition code with $k = 2$ active features. The entropy of this code is

$$\ln \binom{N}{2} = 2 \ln N$$

Here: scenario where we need one dimension per bit of the code!

3 Main Results

We now return to our toy scenario. Let F be a dictionary of N codewords drawn independently and uniformly from the unit sphere in $\mathbb{R}^d$. Let $b(\tau, d, k, N)$ denote the probability that a superposition code for a k -element is *not* accurately recovered by threshold decoding at level τ .

Our first result provides an asymptotic constraint on the dimension d required for $b(\tau, d, k, N)$ to be small.

Proposition 3 *Let $\epsilon > 0$ be arbitrary. Over the regime*

$$d \leq (1 - \epsilon) 2k \ln N,$$

it holds that $\liminf_{N \rightarrow \infty} \inf_{\tau} b(\tau, d, k, N) > 0$.

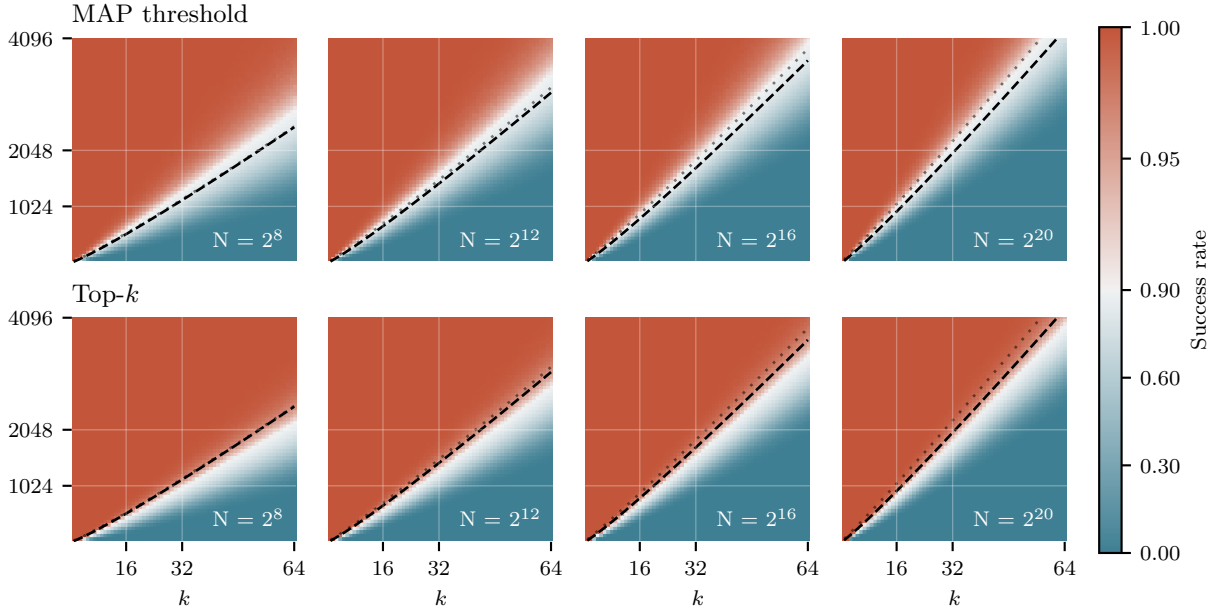


Figure 3: Empirical performance of MAP thresholding and top- k thresholding at decoding a k -element subset of $[N]$ from a d -dimensional superposition code using a random dictionary. The bold line shows the asymptotic requirement $d = (2 + 4\sqrt{\eta} + 2\eta)k \ln N$ predicted by Proposition 4 and the faint line shows the upper bound of $4k \ln(N/k)$.

When $k \geq \epsilon N$ for some $\epsilon > 0$, note that

$$\ln \binom{N}{k} \geq \dots = O(N)$$

while Proposition 3 shows that threshold decoding requires $d = \Omega(k \ln N) = \Omega(N \ln N)$. We conclude that the minimum “dimensions per bit” $\frac{d}{H}$ tends to infinity for large N .

However, it turns out that threshold decoding

Proposition 4 *For $\eta \in (0, 1)$, let $\ln k \leq \eta \ln N$. Then for any $\epsilon > 0$, over the regime*

$$d \geq (1 + \epsilon)(2 + 4\sqrt{\eta} + 2\eta)k \ln N,$$

it holds that $\lim_{N \rightarrow \infty} b(d, k, N) = 0$.

This result has a remarkable interpretation in terms of effective “channel capacity.”

4 Are Random Dictionaries Optimal?

In the last section, we considered the performance of threshold and top- k decodings at recovering a subset from a superposition code with a *random* dictionary F . One natural question is whether these one-step decodings can do better if the dictionary is optimized in some way.

First, recall the

The difficulty of decoding a superposition code for example to reduce the scale of “crosstalk” between distinct codewords.

Proposition 5 (KL bound) *asdf*

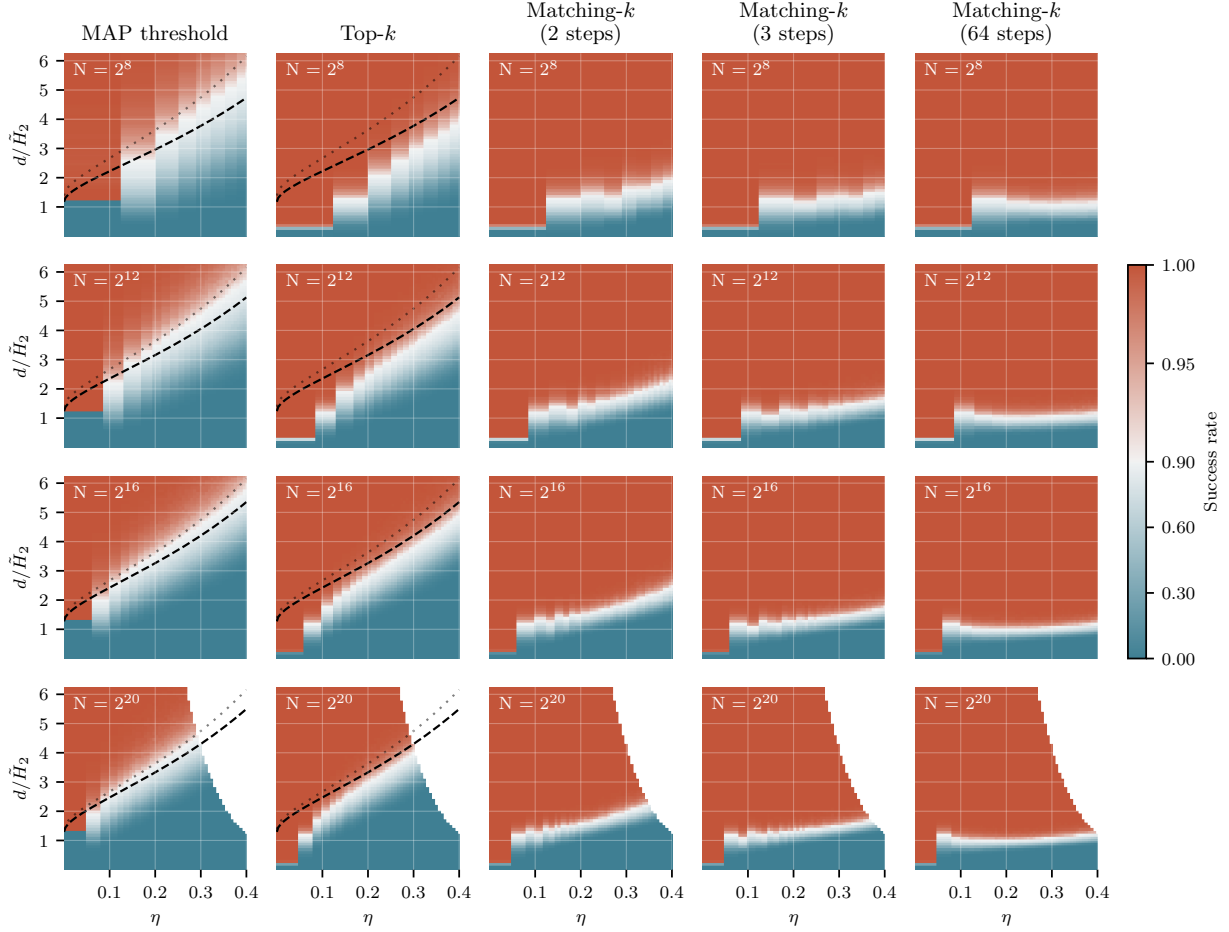


Figure 4: Empirical success rates of different sparse recovery metrics at recovering a superposition code, graphed as a function of inverse bitrate $d/\tilde{H}_2 = d/(k \log_2(eN/k))$ and the sparsity exponent $\eta = \ln k / \ln N$. The bold line shows the “rule of thumb” for MAP thresholding derived from Proposition 4 and the thin line shows the asymptotic lower bound on effective capacity.

Of course, when $d \geq N$, we can make the codewords f_i exactly orthogonal. For this reason, the performance of one-step decodings shown in the left-most column of ?? is much worse than is possible; we never need more than N dimensions to store a latent vector of dimension N . However, when the ratio d/N is small—say, smaller than $1/10$ —we conjecture that optimizing the dictionary gives practically no improvement over a random initialization. Unfortunately, we are not aware of a theoretical justification for this fact.

To understand our conjecture, recall the “crosstalk” terms

$$\xi_i = \sum_{j \neq i} Y_j \langle f_i, f_j \rangle.$$

For each $i \in [N]$, this is a sum of between k and $(k-1)$ numbers drawn without replacement from the sequence $(\langle f_i, f_j \rangle)_{j \neq i}$. Let’s fix the dictionary F and consider the empirical distribution defined by this sequence of $N-1$ numbers. Suppose this distribution has zero mean and variance

$$\gamma_i(F) = \frac{1}{N-1} \sum_{j \neq i} \langle f_i, f_j \rangle^2.$$

When k is moderately large but much smaller than N , we expect the crosstalk ξ_i to behave like a centered Gaussian with variance $k\gamma_i$. Specifically, we expect that the probability of its tail events with respect to the random set Y will be governed by the product $k\gamma_i$. If we assume that tail events for the different variables ξ_i are “sufficiently independent,” we conclude overall that the typical value of $\gamma_i(F)$ is the limiting factor for the reliability of one-step estimates.

A dictionary chosen to make the quantities γ_i uniformly smaller would, in particular, have smaller average squared interference

$$\gamma(F) = \frac{1}{N} \sum_{i=1}^N \gamma_i = \binom{n}{2}^{-1} \sum_{i \neq j} \langle f_i, f_j \rangle^2$$

between distinct codewords. For a random dictionary F , $\gamma(F)$ equals $1/d$ in expectation. Can we decrease this value significantly by optimization?

Using projected gradient descent, we minimized $\gamma(F)$ subject to the constraint of maintaining unit norm codewords. We tested dictionaries with between $N = 64$ and $N = 2^{16} = 65536$ codewords and with codeword dimensions between $d = 16$ and 1024 . In each case, we initialized with a random Rademacher dictionary and optimized to convergence with standard criteria. Our results are shown in Section 4 of ??.

As d approaches N , we find that the optimal value γ_{opt} of $\gamma(F)$ converges to 0, as expected. On the other hand, when $d \ll N$, γ_{opt} is very close to $1/d$, its expected value under a random initialization. For example, with $N = 2^{16}$ (not plotted), the optimal value of $\gamma(F)$ is indistinguishable from $1/d$ on a log-log plot.

Furthermore, we find a striking regularity. Empirically, the ratio $\gamma_{\text{opt}}/d^{-1} = d\gamma_{\text{opt}}$ between the optimal value of γ and its expected value at initialization turns out to be a function of the relative dimension d/N . Since this holds as N ranges over several orders of magnitude, it is natural to believe it may hold in general.

Claim 1 *For given (N, d) , the optimal value of $\gamma(F)$ for a dictionary $F \in \mathbb{R}^{d \times N}$ of unit norm codewords is*

$$\gamma_{\text{opt}}(N, d) = \frac{\kappa(d/N)}{d}$$

for some function κ . Furthermore, $\kappa(r)$ is close to 1 for small values of r .

If true, this means that the values $\gamma_i(F)$ governing the scale of crosstalk suffered by matched filters can’t be made significantly smaller than $1/d$ when $d \leq \epsilon N$ for small ϵ .

We’re not aware of theoretical results in this direction. Note in particular that this is not obviously related to work on sphere packing (see Cohn & Zhao (2014)) since we are interested in the *scale* of the distribution of inner products rather than in maximum values.

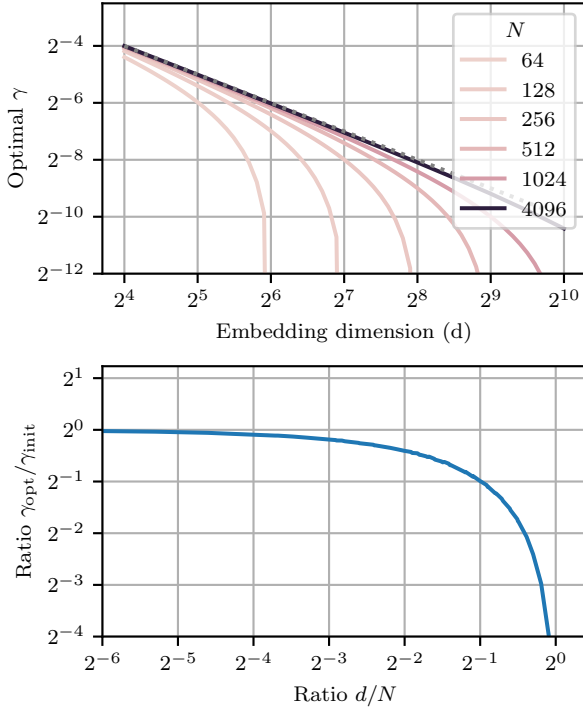


Figure 5: **Top:** The mean squared interference $\gamma(F)$ of a dictionary F obtained by running projected gradient descent to convergence. The dotted line shows $\gamma_{\text{init}} = 1/d$, the mean squared interference attained in expectation by a random initialization. The best interference for $N = 2^{16}$ found by gradient descent (not graphed) is nearly indistinguishable from the dotted line. **Bottom:** A plot of ratio $\gamma_{\text{opt}}/\gamma_{\text{init}}$ by which gradient descent improves γ relative to its expected value at initialization against the ratio d/N between code-word dimension and dictionary size.

5 Conclusions and Future Work

Over the course of computation, it is common for programs to use more memory than was required to store their input data. Similarly, it is natural to expect that the entropy of a “simple” (or “interpretable”) description for the residual vectors within a language model is not bounded by the entropy of language. Methods like sparse autoencoders aim to provide descriptions of these hidden activations, but relatively little is known about the nature of the information is being carried. One basic question is *how much* information can hypothetically be stored in an activation vector using a superposition code.

Previous work showed that sparse autoencoders can help learn interpretable representations of the activity inside a neural network. However, the success of these methods is limited for reasons that are not yet well understood.

In this work, we have identified one point of view that might explain their limited success. In a toy scenario, we showed that the simple estimates these models use to infer sparse representations are much less “efficient,” in an information-theoretic sense, than a simple iterative method. This is true even when the signal to be inferred is extremely sparse. To our knowledge, this kind of explicit, non-asymptotic comparison was not previously available in the literature.

Of course, we do not suggest that the latent signal stored by a typical neural representation is well-modeled as a uniformly random k -sparse subset. However, the “bitrate gap” between one-step estimates and matching pursuit opens a natural question: how much information can neural networks typically encode in their internal activity? Can they, like matching pursuit, read around one bit of mutual information from each neuron? If they can, our findings suggest that sparse autoencoders may be fundamentally unable to decode their representations. Overall, we hope the point of view of coding efficiency helps guide the interpretation of neural representations in future work.

References

- Waheed U. Bajwa, Robert Calderbank, and Sina Jafarpour. Why Gabor frames? Two fundamental measures of coherence and their role in model selection. *Journal of Communications and Networks*, 12(4):289–307, 2010. URL <https://ieeexplore.ieee.org/abstract/document/6388466/>. Publisher: KICS.
- Keith Ball. An Elementary Introduction to Modern Convex Geometry. In Silvio Levy (ed.), *Flavors of Geometry*, Mathematical Sciences Research Institute Publications, pp. 1–58. Cambridge University Press, Cambridge, 1997. ISBN 978-0-521-62962-1. doi: 10.1017/9781009701853.002. URL <https://www.cambridge.org/core/books/flavors-of-geometry/an-elementary-introduction-to-modern-convex-geometry/09EA8B107A9ECA496914D95520C9EAFB>.
- Henry Cohn and Yufei Zhao. Sphere packing bounds via spherical codes. *Duke Mathematical Journal*, 163(10), July 2014. ISSN 0012-7094. doi: 10.1215/00127094-2738857. URL <http://arxiv.org/abs/1212.5966>. arXiv:1212.5966 [math].
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory 2nd Edition*. Wiley-Interscience, Hoboken, N.J, 2006. ISBN 978-0-471-24195-9.
- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse Autoencoders Find Highly Interpretable Features in Language Models, October 2023. URL <http://arxiv.org/abs/2309.08600>. arXiv:2309.08600.
- Sanjoy Dasgupta and Anupam Gupta. An elementary proof of a theorem of Johnson and Lindenstrauss. *Random Structures & Algorithms*, 22(1):60–65, January 2003. ISSN 1042-9832, 1098-2418. doi: 10.1002/rsa.10073. URL <https://onlinelibrary.wiley.com/doi/10.1002/rsa.10073>.
- David L. Donoho and Jared Tanner. Counting Faces of Randomly Projected Polytopes When the Projection Radically Lowers Dimension. *Journal of the American Mathematical Society*, 22(1):1–53, 2009. ISSN 0894-0347. URL <https://www.jstor.org/stable/40587224>. Publisher: American Mathematical Society.
- David L. Donoho, Arian Maleki, and Andrea Montanari. Message-passing algorithms for compressed sensing. *Proceedings of the National Academy of Sciences*, 106(45):18914–18919, November 2009. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.0909892106. URL <https://pnas.org/doi/full/10.1073/pnas.0909892106>.
- M. Elad. *Sparse and redundant representations: from theory to applications in signal and image processing*. Springer, New York, 2010. ISBN 978-1-4419-7010-7 978-1-4419-7011-4. OCLC: ocn646114450.
- Manaal Faruqui, Yulia Tsvetkov, Dani Yogatama, Chris Dyer, and Noah A. Smith. Sparse Overcomplete Word Vector Representations. In Chengqing Zong and Michael Strube (eds.), *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 1491–1500, Beijing, China, July 2015. Association for Computational Linguistics. doi: 10.3115/v1/P15-1144. URL <https://aclanthology.org/P15-1144/>.
- Simon Foucart and Holger Rauhut. Sparse Recovery with Random Matrices. In Simon Foucart and Holger Rauhut (eds.), *A Mathematical Introduction to Compressive Sensing*, pp. 271–310. Springer, New York, NY, 2013. ISBN 978-0-8176-4948-7. doi: 10.1007/978-0-8176-4948-7_9. URL https://doi.org/10.1007/978-0-8176-4948-7_9.
- Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders, June 2024. URL <http://arxiv.org/abs/2406.04093>. arXiv:2406.04093 [cs].
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *Advances in Neural Information Processing Systems*, 35:17359–17372, 2022.

- Anh Nguyen, Jason Yosinski, and Jeff Clune. Multifaceted Feature Visualization: Uncovering the Different Types of Features Learned By Each Neuron in Deep Neural Networks, May 2016. URL <http://arxiv.org/abs/1602.03616>. arXiv:1602.03616 [cs].
- Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom In: An Introduction to Circuits. *Distill*, 5(3):10.23915/distill.00024.001, March 2020. ISSN 2476-0757. doi: 10.23915/distill.00024.001. URL <https://distill.pub/2020/circuits/zoom-in>.
- David E. Rumelhart, James L. McClelland, and AU. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition: Foundations*. The MIT Press, 1986. ISBN 978-0-262-29140-8. doi: 10.7551/mitpress/5236.001.0001.
- Lee Sharkey, Bilal Chughtai, Joshua Batson, Jack Lindsey, Jeff Wu, Lucius Bushnaq, Nicholas Goldowsky-Dill, Stefan Heimersheim, Alejandro Ortega, Joseph Bloom, Stella Biderman, Adria Garriga-Alonso, Arthur Conmy, Neel Nanda, Jessica Rumbelow, Martin Wattenberg, Nandi Schoots, Joseph Miller, Eric J. Michaud, Stephen Casper, Max Tegmark, William Saunders, David Bau, Eric Todd, Atticus Geiger, Mor Geva, Jesse Hoogland, Daniel Murfet, and Tom McGrath. Open Problems in Mechanistic Interpretability, January 2025. URL <http://arxiv.org/abs/2501.16496>. arXiv:2501.16496 [cs].
- Keigo Takeuchi. Generalized Approximate Message-Passing for Compressed Sensing with Sublinear Sparsity, September 2024. URL <https://arxiv.org/abs/2409.06320v2>.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Calum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread*, May 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- Simon Thorpe. Local vs. Distributed Coding. *Intellectica*, 8(2):3–40, 1989. doi: 10.3406/intel.1989.873. URL https://www.persee.fr/doc/intel_0769-4113_1989_num_8_2_873. Publisher: Persée - Portail des revues scientifiques en SHS.
- Zeyu Yun, Yubei Chen, Bruno Olshausen, and Yann LeCun. Transformer visualization via dictionary learning: contextualized embedding as a linear superposition of transformer factors. In Eneko Agirre, Marianna Apidianaki, and Ivan Vulić (eds.), *Proceedings of Deep Learning Inside Out (DeeLIO): The 2nd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, pp. 1–10, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.deelio-1.1. URL <https://aclanthology.org/2021.deelio-1.1/>.
- Changwan Zhang and Yue Wang. A sample survey study of poly-semantic neurons in deep CNNs. In *International Conference on Computer Graphics, Artificial Intelligence, and Data Processing (ICCAID 2022)*, volume 12604, pp. 849–855. SPIE, May 2023. doi: 10.1117/12.2674650.

A Basic Results on Random Vectors

The following is a well-known estimate for the tail of a Gaussian.

Proposition 6 *When Z is a unit Gaussian, we have*

$$\ln P(Z \geq a) = -\frac{1}{2}a^2 - \ln a - \frac{1}{2} \ln 2\pi + o(1).$$

for $a \rightarrow +\infty$.

In fact, the leading-order term $-\frac{1}{2}a^2$ is an upper bound on $\ln P(Z \geq a)$. One simple way to establish a similar estimate on the tail probabilities of a variable X is via the so-called Chernoff inequalities

$$\ln P(X \geq a) \leq -\lambda a + K_X(\lambda)$$

where $K_X(\lambda) = \ln E \exp(\lambda X)$ is the cumulant generating function. For example, for a unit Gaussian Z we have $K_Z(\lambda) = \lambda^2/2$, which gives

$$\ln P(Z \geq a) \leq -\lambda a + \frac{1}{2}\lambda^2.$$

Minimizing with respect to λ produces the inequality

$$\ln P(Z \geq a) \leq -\frac{1}{2}a^2.$$

Now, consider the inner product of two random vectors X, Y drawn independently and uniform from the unit sphere S^{d-1} in d dimensions. Write $b(d, t)$ for the probability that $\langle X, Y \rangle \geq t$. This number can also be viewed as the volume of the spherical cap

$$C_{d,t} = \{v \in S^{d-1} : v_1 \geq t\}$$

under the probability measure on S^{d-1} . Using the fact that we can construct a uniform distribution on the unit sphere by normalizing a Gaussian random vector, a simple line of reasoning can establish that $b(d, t) \leq (1 + o(1)) \exp(-\frac{1}{2}dt^2)$ using Chernoff inequalities. More precisely, it will be useful to have the following upper and lower bounds.

Proposition 7 *Where X and Y are independently drawn from the uniform distribution on the unit sphere in $\mathbb{R}^d$, let $b(d, t) = P(\langle X, Y \rangle \geq t)$. Then for all $t \in (0, 1)$,*

$$\exp\left(-\frac{dt^2}{2(1-t^2)}\right) \leq b(d, t) \leq \exp\left(-\frac{dt^2}{2}\right)$$

This is a corollary of some results from lecture Ball (1997).

Proposition 8 *Let $d > 2\epsilon^{-2}(2 \ln N + \ln p^{-1})$, and let*

$$\{F_1, \dots, F_N\} \subseteq \{-1/\sqrt{d}, 1/\sqrt{d}\}^d$$

be random vectors with independent, uniformly distributed entries. Then $|\langle F_i, F_j \rangle| < \epsilon$ for all $i \neq j$ with probability at least $(1 - p)$.

Proof 1 *Each inner product $I = \langle F_i, F_j \rangle$ is distributed like a sum of d Rademacher variables scaled by $1/d$. By the Chernoff bound above, we have that*

$$\ln P(I \geq \epsilon) = P(X_d/d \geq \epsilon) \leq -\frac{d^2\epsilon^2}{2d} = -\frac{1}{2}d\epsilon^2.$$

By symmetry $P(I \geq \epsilon) = P(I \leq -\epsilon)$, and so by a union bound

$$\ln P(|\langle F_i, F_j \rangle| \geq \epsilon) \leq \ln(2P(I \geq \epsilon)) \leq -\frac{1}{2}d\epsilon^2 + \ln 2.$$

To conclude that $|\langle F_i, F_j \rangle| < \epsilon$ for all $\binom{N}{2} < N^2/2$ pairs of vectors with probability at least $1 - p$ by a union bound, it suffices that

$$\begin{aligned} -\frac{1}{2}d\epsilon^2 + \ln 2 &\leq \ln \frac{p}{N^2/2} \\ &= -2 \ln N + \ln 2 + \ln p, \end{aligned}$$

which is equivalent to the condition on d above.

The interested reader should also compare this result to the Johnson-Lindenstrauss lemma, which is proved in a very similar way. (See Dasgupta & Gupta (2003) for a proof, or the last section of Foucart & Rauhut (2013) for a discussion of the JL lemma with some broader context.)

B Matched Filters

Consider the problem of inferring a scalar S from the sum

$$X = Sf + Z$$

where $f \in \mathbb{R}^n$ and Z is a Gaussian variable independent from S . Suppose for simplicity that Z has non-singular covariance Σ , so that $-\ln p(z) = 1/2 \|z\|_\Sigma^2$ where

$$\|z\|_\Sigma^2 = z^T \Sigma^{-1} z.$$

Then a routine calculation shows that

$$\begin{aligned} -\ln p(S = s | X = x) \\ = C(x) - \ln p(s) + \frac{1}{2} \left(s - \frac{\langle f, x \rangle_\Sigma}{\|f\|_\Sigma^2} \right)^2 \|f\|_\Sigma^2 \end{aligned} \quad (5)$$

where $C(x)$ is a constant depending only on x and $\langle -, - \rangle_\Sigma$ is the inner product associated with the norm $\|-\|_\Sigma$. In particular, the distribution of S conditional on X is only a function of the inner product $\langle f, X \rangle_\Sigma$. The **matched filter** for S is the linear function

$$\lambda(x) = \frac{\langle f, x \rangle_\Sigma}{\|f\|_\Sigma^2},$$

and can be understood as providing the maximum likelihood estimate for S .

In general, the quality of a linear filter λ' as an estimate for S can be measured by the signal-to-noise ratio

$$\rho(\lambda') = \frac{(\lambda'(f))^2}{\text{Var}_Z \lambda'(Z)}.$$

The maximum possible signal-to-noise ratio is $\|f\|_\Sigma^2$, which is achieved by λ . Given an improper uniform prior on S , Equation (5) shows the posterior on S conditional on X is a Gaussian with mean $\lambda(X)$ and precision $\|f\|_\Sigma^2$.

C Estimates for the Binomial Coefficient

To estimate $\ln \binom{N}{k}$, it is helpful to first remember the elementary inequalities

$$\left(\frac{N}{k} \right)^k \leq \binom{N}{k} \leq \left(\frac{eN}{k} \right)^k.$$

Taking logarithms gives

$$k \ln \left(\frac{N}{k} \right) \leq \ln \binom{N}{k} \leq k \ln \left(\frac{eN}{k} \right),$$

and so

$$\ln \binom{N}{k} = k(\ln N - \ln k + O(1)).$$

In the sublinear regime $\ln k \sim \eta \ln N$, this proves that

$$\ln \binom{N}{k} \sim (1 - \eta)k \ln N.$$

This is the rough approximation we need for our asymptotic results. However, when $k^2 \ll N$, it is also useful to have access to the finer approximation

$$\ln \binom{N}{k} = k(\ln N - \ln k + 1 + O(k/N)).$$

In terms of η , this means that

$$\ln \binom{N}{k} = (1 - \eta)k \ln N + k + o(1).$$

for large N so long as $\eta < 1/2$.

One heuristic justification for this finer estimate is to consider a random subset $Y \subseteq [N]$ where each element is included independently with probability $s = k/N$. Then, where

$$h(s) = -s \ln s - (1 - s) \ln(1 - s) = -s \ln s + s + O(s^2)$$

is the binary entropy function, the entropy of Y is

$$\begin{aligned} H(Y) &= h(s)N = -sN \ln s + sN + O(s^2 N) \\ &= k \ln \left(\frac{eN}{k} \right) + O\left(\frac{k^2}{N} \right). \end{aligned}$$

We expect that for large N the number of elements in Y concentrates sharply around k , and so $H(Y)$ should be close to $\ln \binom{N}{k}$.

Indeed this is true. One simple approach is to use Stirling's approximation

$$\ln n! = n \ln n - n + \frac{1}{2} \ln(2\pi n) + O(n^{-1})$$

to derive

$$\begin{aligned} \ln \binom{N}{k} &= \ln N! - \ln k! - \ln(N - k)! \\ &= k \ln \frac{N}{k} + (N - k) \ln \left(1 + \frac{k}{N - k} \right) + O\left(\frac{1}{N} \right). \end{aligned}$$

For $k \ll N$, we have

$$\ln \left(1 + \frac{k}{N - k} \right) = \frac{k}{N} + O\left(\frac{k^2}{N^2} \right).$$

Overall this proves that

$$\ln \binom{N}{k} = k \ln \left(\frac{eN}{k} \right) + O\left(\frac{k^2}{N} \right).$$